

Programming Techniques 2026–2027

Lecture 1: Introduction and Version Control

Hannu Parviainen

Universidad de la Laguna

September 16, 2026

Programming Techniques 2026–2027

- ▶ Lecturer: Hannu Parviainen (ULL, IAC)
- ▶ Language: English
- ▶ Contact: hparviai@ull.edu.es
- ▶ Schedule: Mondays and Wednesdays 15:00–17:00
- ▶ Dates: 16 September – November 2026
- ▶ Location: CCA
- ▶ Structure: theory + exercises
- ▶ Reading: Fortran book, Pro Git, old lecture notes

Programming Techniques 2026–2027

Assessment options:

1. Two equally weighted exercises, no final exam
 - ▶ Both exercises can be done over the period of several weeks
 - ▶ GitHub is used to follow and return the exercises, as well as to discuss and share ideas
 - ▶ Scoring is detailed separately for each exercise. However, the main criteria are that the code works, is readable and is well-documented. Interaction and involvement in GitHub is also important.
2. Final exam: writing a fully working parallelised Fortran program in 4 h (paper + pen)
 - ▶ Nobody has ever chosen this

Course Topics

- ▶ Programming techniques in astronomy and astrophysics
- ▶ Modern Fortran
 - ▶ Data types
 - ▶ Flow control structures
 - ▶ Modules, subroutines and functions
 - ▶ Dynamic memory management (allocatable arrays, pointers)
 - ▶ Data structures: linked lists and trees
- ▶ Parallelisation: OpenMP
- ▶ Distributed computing: MPI
- ▶ Version control: Git + GitHub
- ▶ Makefiles

Not Course Topics

IDEs

- ▶ Course teaches programming, not how to use a specific Integrated Development Environment
- ▶ The code can be written using a text editor and compiled using command-line commands
- ▶ ...however, a good IDE can make the course way less painful
- ▶ Good options: VSCode (with the *Modern Fortran* extension and the `fortls` language server), Code::Blocks, Emacs (old-school)
- ▶ Check the list from fortran-lang.org

Lecturer: Hannu Parviainen

- ▶ Research: exoplanets, Bayesian statistics, scientific computing, radiative transfer
- ▶ BASIC, PASCAL, C, and C++ 1991–2004
- ▶ Fortran as main language 2004–2017
- ▶ Now: Python + Numba + OpenCL + JAX
- ▶ Offices at IAC and ULL



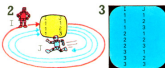
Tricks with loops

Here are some more programs using loops. Below you can find out how you can use loops within loops to repeat several things at the same time. These are called nested loops.

1 Nested loops

```
5 PRINT "I, J"
10 FOR I=1 TO 3
20 FOR J=1 TO 3
30 PRINT I, J
40 NEXT J
50 NEXT I
60 END
```

This program has an I loop and a J loop. The J loop is nested inside the I loop and for each time that the I loop is carried out, the J loop is repeated three times, printing



out the value for J each time. The picture above shows the result of this program. The commas spaced the figures out like this.

Computer clock

```
10 CLS
20 LET M=0
30 LET S=0
40 FOR M=0 TO 59
50 FOR S=0 TO 59
60 PRINT M, ":", S
70 CLS
80 NEXT S
90 NEXT M
100 END
```

Inside a computer there is an electronic "clock" which uses the rhythm for all the computer's work. The clock pulses at between one and four million pulses a second. This program makes the computer behave like a digital clock. It has nested loops, one to count the seconds and one to count the minutes.



The seconds loop is carried out 59 times for each minute loop. If you try this program on a computer it might run very fast at first. You need to put in an extra "delay loop", then set it by changing the figure in the loop as your computer clock "ticks" at the same rate as a real one.

```
10 FOR I=1 TO 4
20 FOR J=1 TO 4
30 PRINT I, J
40 NEXT J
50 NEXT I
60 END
```

Both parts of a nested loop must be inside the other line.

Random number tester

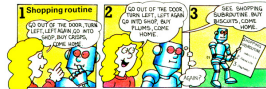
```
10 FOR I=1 TO 1000
20 LET R=INT(RND*(1+E+1))
30 IF R=1 THEN LET A=A+1
40 IF R=2 THEN LET B=B+1
50 IF R=3 THEN LET C=C+1
60 IF R=4 THEN LET D=D+1
70 IF R=5 THEN LET E=E+1
80 IF R=6 THEN LET F=F+1
90 NEXT I
100 PRINT "FINISHED"
110 PRINT A, B, C
120 PRINT D, E, F
130 END
```

28

*On some computers, e.g. ZX81 and BBC micro, you need some extra lines at the beginning of the program to set each variable to 0.

Subroutines

A subroutine is a sort of mini-program within a program. It carries out a particular task, such as adding numbers or keeping a score, and you can send the computer to it whenever you want this task carried out. This saves writing out the program lines each time and makes the program shorter and easier to read and type into the computer.



Suppose you had a robot helper whom you could program to run errands for you. If you wanted something from the shop you would have to give it precise instructions telling it how to get there.

Each time you wanted the robot to buy something you would have to give it the same instructions. It would be much simpler to give the robot a shopping subroutine and tell it to refer to it each time.

4 Shopping program

```
10 PRINT "WHAT DO YOU WANT FROM THE SHOP"
20 INPUT XS
30 GOSUB 100
40 PRINT "ANYTHING ELSE"
50 INPUT MS
60 IF MS="YES" THEN GOTO 10
70 STOP

100 REM: SHOP SUBROUTINE
110 PRINT "GO OUT, TURN LEFT"
120 PRINT "LEFT AGAIN, ENTER SHOP"
130 PRINT "BUY ",XS," COME HOME"
140 RETURN
```



Line 30 sends the computer to the first line of the subroutine.

You need the word STOP at the end of the main program to stop the computer carrying on into the subroutine.

It is useful to label a subroutine with a REM line so you know what it is for.

This sends the computer back to line 40, the line after GOSUB.

In BASIC, to tell the computer to go to a subroutine you use the word GOSUB with the word RETURN at the end of the subroutine. GOSUB should be followed by the number of the first line of the subroutine. RETURN does not need a line

number. The computer automatically goes back to the instruction after the one where it left the main part of the program. You can send the computer to a subroutine anywhere in the program as many times as you like.

Basic Workflow

1. Fork the official course repository on GitHub into your own account
2. Clone your forked repository to your local computer
3. Work on exercises in your own computer
4. Push changes back to your fork on GitHub
5. Open pull requests to merge your work to the main course repository

Introduction to Fortran

Fortran

- ▶ Compiled programming language (vs. interpreted like Python)
- ▶ Developed in the 1950s for scientific computing
- ▶ Very good for numerically heavy problems in physics
- ▶ Not really good for much else... but works well together with Python!
- ▶ Major versions: FORTRAN 77, Fortran 90/95, 2003, 2008, 2018, 2023

Compiled vs Interpreted Languages

Compiled (Fortran, C, C++)

- ▶ Source → machine code via compiler
- ▶ Produces executable before running
- ▶ Optimised for target CPU
- ▶ Requires a compiler

Interpreted (Bash, Python, R, Matlab)

- ▶ Source executed by an interpreter at run time
- ▶ No separate compilation step
- ▶ Requires interpreter to run

Performance

Compiled

- ▶ Very fast execution
- ▶ Runs as native machine code, with no interpreter in between
- ▶ Speed still depends on compiler optimisation (e.g., -O2, -O3)

Interpreted

- ▶ Slower due to interpreter overhead
- ▶ Pure-Python loops are often 10–100× slower than compiled code
- ▶ NumPy, Numba, and JAX can close most of the gap

Development Speed

Compiled

- ▶ Slower cycle: edit → compile → run
- ▶ More boilerplate code

Interpreted

- ▶ Rapid iteration: edit → run
- ▶ Shorter programs, easier syntax

Portability

Compiled

- ▶ Standard-conforming source code is portable
- ▶ ...but the executable must be rebuilt for each OS and CPU architecture

Interpreted

- ▶ The code can be run with any setup (if the interpreter exists)

Error Handling

Compiled

- ▶ Many errors caught at compile time
- ▶ Type mismatches, missing variables flagged early
- ▶ Some errors still happen at run time (e.g., out-of-bounds array access, which is *not* checked by default)
- ▶ During development, compile with `-Wall -fcheck=all`

Interpreted

- ▶ Errors appear at runtime
- ▶ Flexible, but can fail late

Typical Use Cases

Compiled

- ▶ Heavy numerical simulations
- ▶ High-performance computing
- ▶ Long-running production code

Interpreted

- ▶ Data analysis and exploration
- ▶ Rapid prototyping
- ▶ Teaching, scripting, glue code

Fortran: First Steps

- ▶ Program code inside a program ...
end program block
- ▶ Always use implicit none
- ▶ Simple printing: print *, ...

```
program hello  
  implicit none  
  print *, "Hello world!"  
end program hello
```

Compiling Fortran

- ▶ Source code file → executable program
- ▶ Free compilers: GNU gfortran, LLVM flang, Intel ifx, NVIDIA nvfortran, AMD AOCC
- ▶ Vendor compilers optimise for their own hardware, but gfortran is usually enough (and is what we use)

```
$ gfortran -o hello hello.f90  
$ ./hello
```

Fortran Source Files

- ▶ Use suffix `.f90` for modern Fortran
- ▶ Avoid version-specific suffixes (`.f03`, `.f08`, ...)

Version Control

Version Control: What It Is and Why We Need It

- ▶ Tracks changes in code, documents, and data
- ▶ Allows you to:
 - ▶ Revert to earlier versions
 - ▶ Compare changes over time
 - ▶ Work on features without breaking main code
- ▶ Essential for collaboration
- ▶ Prevents the “final_v2_really_final.f90” problem

Version Control Concepts

- ▶ **Repository**: a project's files plus their complete history
- ▶ **Commit**: snapshot of project at a given time
- ▶ **Branch**: parallel line of development
- ▶ **Merge**: combine changes from different branches
- ▶ **Remote**: another copy of the repository (e.g., on GitHub) that you exchange commits with

Git is by far the most widely used system today. Older systems, such as Subversion (SVN) and Mercurial, are still found in some older projects.

What is Git?

Distributed Version Control System (VCS)

- ▶ Tracks changes in the source code and helps coordinate work among programmers.
- ▶ Designed to handle everything from small to very large projects quickly and efficiently.
- ▶ Allows multiple developers to work on a project simultaneously without overwriting each other's changes.

Distributed?

- ▶ Each user has a full copy of the repository.
- ▶ Allows for a more flexible workflow than centralised VCSs.
- ▶ Committing, branching, and browsing history are fast and do not require a network connection.

Why Use Git?

- ▶ **Collaboration:** Streamlines teamwork on projects.
- ▶ **Backup and Restore:** Safeguards code with version history.
- ▶ **Track History:** Allows you to see what changes were made and when.
- ▶ **Branching and Merging:** Supports multiple development lines.

Installing Git

Windows

Install via Git for Windows.

Mac

Install the Xcode command-line tools with `xcode-select --install`, or use Homebrew:
`brew install git`.

Linux

Install via package manager using `sudo apt install git` or equivalent.

Configuring Git

Set your username, email, and default branch name:

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
```

- ▶ The name and email are used to attribute commits to you.
- ▶ Use the same email as in your GitHub account so that GitHub links the commits to your profile.
- ▶ Without the last setting, new repositories may still start with a branch called `master` instead of `main`.
- ▶ The `--global` flag applies settings for all repositories.

Initializing a Repository

Initialize a new Git repository in the current directory:

```
$ git init
```

- ▶ Creates a new `.git` subdirectory that will store the history.
- ▶ Nothing is tracked yet: files must be added with `git add` and committed with `git commit`.

Checking Status

Check the status of your working directory and staging area:

```
$ git status
```

- ▶ Shows untracked files and modified tracked files.
- ▶ Indicates changes that are staged for commit.

How Git Tracking Works

- ▶ **Working Directory:** Your local filesystem where you modify files.
- ▶ **Staging Area (Index):** A temporary area where you add changes to prepare for a commit.
- ▶ **Repository (History):** The database where commits are stored.
- ▶ **Tracking Changes:**
 - ▶ Git monitors changes in tracked files.
 - ▶ Untracked files are not monitored until added.
- ▶ **Lifecycle of a File:**
 1. Modify files in the working directory.
 2. Stage changes using `git add`.
 3. Commit changes to the repository with `git commit`.

Staging vs. Committing Changes

- ▶ **Staging Changes** (`git add`):
 - ▶ Prepares selected changes for the next commit.
 - ▶ Allows you to review and group changes.
- ▶ **Committing Changes** (`git commit`):
 - ▶ Records the staged changes into the repository history.
 - ▶ Creates a new commit object with a unique ID.
 - ▶ Includes a commit message describing the changes.
- ▶ **Key Differences:**
 - ▶ **Staging:** Prepares changes, but does not save them to history.
 - ▶ **Committing:** Records the staged changes in the history.

Staging Changes with `git add`

Add files to the staging area:

```
$ git add filename  
$ git add .
```

- ▶ `git add filename`: Stages a specific file.
- ▶ `git add .`: Stages *all* changes (new, modified, and deleted files) in the current directory and its subdirectories.
- ▶ **Staging**: Prepares changes to be included in the next commit.
- ▶ Check `git status` before committing so that you do not add files by accident!

Ignoring Files with .gitignore

A .gitignore file lists files that Git should never track:

```
# Compiled objects and modules
*.o
*.mod
# Executables
hello
*.exe
```

- ▶ Keep build products (object files, modules, executables) and large data files out of the repository.
- ▶ These files can always be regenerated from the source code.
- ▶ The .gitignore file itself is committed like any other file.

Committing Changes with `git commit`

Commit the staged changes to the repository:

```
$ git commit -m "Commit message"
```

- ▶ Records a snapshot of the staging area.
- ▶ **Commit Message:** Describes the changes made.

Inspecting History and Changes

```
$ git log --oneline      # list commits, newest first
$ git diff              # unstaged changes
$ git diff --staged     # staged changes (what will be committed)
$ git show <commit>    # contents of a single commit
```

- ▶ Use `git diff` before `git add` to review what you changed.
- ▶ Each commit is identified by a hash (e.g., `9ce14a9`); a short prefix is usually enough.

Understanding Branches

► What is a Branch?

- A lightweight, movable pointer to a commit that advances as you commit.
- Creating one is instantaneous, so branches are cheap to use.
- Allows you to work on different features independently.

► Why Use Branches?

- Isolate development work without affecting the main codebase.
- Facilitate collaboration by allowing multiple features to be developed simultaneously.

► Default Branch: Typically named `main` (older repositories: `master`).

Creating and Switching Branches

Create a new branch and switch to it:

```
$ git branch new-feature  
$ git switch new-feature
```

Or combine both steps:

```
$ git switch -c new-feature
```

- ▶ `git branch new-feature`: Creates a new branch named `new-feature`.
- ▶ `git switch new-feature`: Switches to the `new-feature` branch.
- ▶ `git switch -c new-feature`: Creates and switches to `new-feature` in one command.
- ▶ Older guides use `git checkout` (and `git checkout -b`) for the same purpose; it still works.

Merging Branches

Merge changes from new-feature branch into main:

```
$ git switch main  
$ git merge new-feature
```

- ▶ `git switch main`: Switches to the main branch.
- ▶ `git merge new-feature`: Merges new-feature into main.
- ▶ **Merging**: Combines changes from one branch into another.
- ▶ **Conflicts**: If both branches changed the same lines, Git stops and marks the conflicting lines in the files. Edit the files to resolve the conflict, then `git add` and `git commit`.

What is GitHub?

- ▶ **Code Hosting Platform:** Hosts Git repositories online.
- ▶ **Facilitates Collaboration:** Tools for team communication and coordination.
- ▶ **Additional Features:**
 - ▶ Issue tracking.
 - ▶ Pull requests for code reviews.
 - ▶ Wiki pages and documentation.

Setting Up GitHub

- ▶ **Create an Account:** Sign up at github.com.
- ▶ **Enable Two-Factor Authentication (2FA):** Required by GitHub for code contributors (e.g., use an authenticator app).
- ▶ **Set Up Authentication for Git:**
 - ▶ Your GitHub password *cannot* be used for `git push` or `git pull`.
 - ▶ Use an SSH key (next slide) and the SSH repository URLs (`git@github.com:...`).
 - ▶ Alternative: the GitHub CLI, `gh auth login`, sets everything up interactively.

Setting Up an SSH Key

```
# 1. Generate a key pair (press Enter to accept the default location)
$ ssh-keygen -t ed25519 -C "you@example.com"

# 2. Print the public key and copy it
$ cat ~/.ssh/id_ed25519.pub

# 3. On GitHub: Settings > SSH and GPG keys > New SSH key, paste the key

# 4. Test the connection
$ ssh -T git@github.com
```

- Only the *public* key (.pub) is shared. Never share the private key.

Connecting a Local Repository to GitHub

Add a remote repository and push changes:

```
$ git remote add origin git@github.com:username/repo.git  
$ git push -u origin main
```

- ▶ `git remote add origin`: Links your local repo to GitHub under the name `origin`.
- ▶ `git push -u origin main`: Pushes your local `main` branch to GitHub.
- ▶ **The `-u` Flag**: Makes `origin/main` the upstream of your local `main`, so that a plain `git push` or `git pull` works afterwards.
- ▶ A repository created with `git clone` already has `origin` set up.

Pushing and Pulling

```
$ git push      # send your new commits to the remote
$ git pull      # fetch new commits from the remote and merge them
```

- ▶ `git push` is rejected if the remote has commits you do not have: `git pull` first, then push again.
- ▶ Pull regularly to avoid large merge conflicts.

Collaborating on GitHub

- ▶ **Forking Repositories:** Create a personal copy of someone else's project *on GitHub*.
- ▶ **Cloning Repositories:** Download a repository to your local machine.
- ▶ **Pull Requests:** Propose changes to a repository.
- ▶ **Code Reviews:** Collaboratively review code changes before merging.
- ▶ **Issue Tracking:** Report bugs or request features.

The Fork Workflow

- ▶ **upstream**: the original course repository (hpparvi/ull_tp_2627)
- ▶ **origin**: your fork on GitHub (your-username/ull_tp_2627)
- ▶ **local**: the clone on your computer

You push to origin and open pull requests from origin to upstream. To get new course material, add the upstream remote once and pull from it:

```
$ git remote add upstream https://github.com/hpparvi/ull_tp_2627.git
$ git pull upstream main      # get the latest course material
$ git push origin main       # update your fork on GitHub
```

Alternatively, use the **Sync fork** button on your fork's GitHub page and then `git pull`.

Best Practices

- ▶ **Write Meaningful Commit Messages:** Clearly describe what changes were made.
- ▶ **Keep Commits Small and Focused:** Easier to review and understand.
- ▶ **Regularly Pull Updates:** Keep your local repository up-to-date with the remote.
- ▶ **Use Branches for New Features:** Isolate development work.
- ▶ **Do Not Commit Build Products:** Use `.gitignore` for object files, modules, and executables.
- ▶ **Avoid Committing Sensitive Information:** Don't commit passwords or API keys. Deleting the file in a later commit does *not* remove it from the history, so treat a committed secret as leaked and change it.

Preliminaries

Setup a Fortran Compiler

- ▶ Optional: set up a Conda environment
- ▶ Install gfortran
- ▶ Optional: install VSCode or another IDE

Setup Git & GitHub

- ▶ Install and configure Git (if not already done)
- ▶ Set up a GitHub account with 2FA
- ▶ Set up an SSH key and add it to GitHub

Preliminaries

Setup Course

- ▶ On GitHub: click “Fork” on the course repository page:
https://github.com/hpparvi/ull_tp_2627
- ▶ On your computer: clone your fork

```
$ git clone git@github.com:your-username/ull_tp_2627.git  
$ cd ull_tp_2627
```

- ▶ Add the course repository as upstream and verify your remotes

```
$ git remote add upstream https://github.com/hpparvi/ull_tp_2627.git  
$ git remote -v
```

First Exercise

Follow the “First steps” in `students/README.md`:

1. Create your own directory `students/<initials>` (e.g., `students/ph`)
2. Create the subdirectories `sandbox/101`, `course_exercise_1`, and `course_exercise_2`
3. Add empty `README.md` files to the course exercise directories
4. Add your name and directory to the table in `students/README.md`
5. Stage, commit, and push the changes to your fork
6. On GitHub, open a pull request to the course repository